Claude AI 개인 비서 구축 가이드

- □ Claude AI 개인 비서 구축 완전 가이드
  - □ 목차
  - 1. 전체 구조 이해
  - 2. AWS EC2 서버 생성
    - 2-1. AWS 계정 및 EC2 생성
    - 2-2. SSH 접속
    - 2-3. 기본 패키지 설치
  - 3. Claude Code 설치 및 연동
    - 3-1. Node.js 최신 버전 설치 (필수)
    - 3-2. Claude Code 설치
    - 3-3. Claude 인증
    - 3-4. cokacdir 설치 (파일 전송 도구)
  - 4. Telegram 봇 설정
    - 4-1. 봇 생성
    - 4-2. Chat ID 확인
    - 4-3. 설정 파일 생성
    - 4-4. 봇 테스트
  - 5. GitHub 연동
    - 5-1. GitHub Personal Access Token (PAT) 생성
    - 5-2. Git 전역 설정
    - 5-3. 레포 생성 및 연결
    - 5-4. .gitignore 설정 (보안 필수!)
  - 6. Moltbook 연동
    - 6-1. Moltbook API 키 발급
    - 6-2. 설정 파일 생성
    - 6-3. Moltbook 포스트 작성 Python 함수
  - 7. 아침/오후/저녁 리포트 자동화
    - 7-1. 모닝 리포트 (`morning_report.py`)
    - 7-2. 크론잡 등록
  - 8. 포트폴리오 모니터링

    - 8-1. 포트폴리오 정의
    - 8-2. 기술적 분석 함수
    - 8-3. 크론잡 (평일 3회)
  - 9. 7엔진 마켓 인텔리전스 시스템
    - 9-1. 레포 구조
    - 9-2. GitHub 레포 생성
    - 9-3. run_all.py 핵심 구조
  - 10. EC2 자동 시작/종료 스케줄링
    - 10-1. IAM 역할 생성
    - 10-2. AWS CLI 설치
    - 10-3. EventBridge Scheduler로 자동화
    - 10-4. 전체 스케줄 구성 예시
  - 11. 운영 팁 & 트러블슈팅
    - □ 비용 절감 팁
    - □ 자주 발생하는 오류
    - □ 전체 디렉토리 구조
    - □ 빠른 시작 체크리스트
  - □ 참고 자료

□ Claude AI 개인 비서 구축 완전 가이드

> AWS EC2 + Claude Code + Telegram + GitHub + Moltbook 연동

대상: Claude 유료 구독자 (Pro / Max 플랜) **난이도**: □□□ (중급) **소요 시간**: 약 2~3시간 **월 비용**: AWS EC2 t3.micro 약 $10~15 (스케줄링으로 절감 가능)

□ 목차

1. 전체 구조 이해
2. AWS EC2 서버 생성

3. Claude Code 설치 및 연동
4. Telegram 봇 설정
5. GitHub 연동
6. Moltbook 연동
7. 아침/오후/저녁 리포트 자동화
8. 포트폴리오 모니터링
9. 7엔진 마켓 인텔리전스 시스템
10. EC2 자동 시작/종료 스케줄링
11. 운영 팁 & 트러블슈팅

---

1. 전체 구조 이해

```
┌──────────────────────────────────────────────────────────┐
│                    전체 시스템 구조                       │
│                                                          │
│  ┌──────────┐    ┌──────────────┐    ┌──────────────┐   │
│  │EventBridge│→  │  AWS EC2     │ →  │  Telegram    │   │
│  │스케줄러  │    │(t3.micro)    │    │   Bot        │   │
│  └──────────┘    │              │    └──────────────┘   │
│                  │ Claude Code  │                       │
│  ┌──────────┐    │  + Python    │    ┌──────────────┐   │
│  │  Cron    │ →  │              │ →  │   GitHub     │   │
│  │  Jobs    │    │ 7개 분석엔진 │    │   레포       │   │
│  └──────────┘    └──────────────┘    └──────────────┘   │
│                          ↕                               │
│                  ┌──────────────┐                        │
│                  │  Moltbook    │                        │
│                  │  API 연동    │                        │
│                  └──────────────┘                        │
└──────────────────────────────────────────────────────────┘

자동화 흐름:
```

```
새벽 01:50 → EC2 시작 → 02:00 시장분석+GitHub push → 02:30 EC2 종료
아침 07:50 → EC2 시작 → 08:00 모닝리포트 → 09:00~15:00 포트폴리오 모니터링 → 종료
```

---

2. AWS EC2 서버 생성

2-1. AWS 계정 및 EC2 생성

1. **AWS 콘솔** → EC2 → 인스턴스 시작
2. 설정값:

```
AMI       : Ubuntu Server 24.04 LTS (무료)
인스턴스  : t3.micro (월 ~$8, 프리티어 t2.micro도 가능)
스토리지  : 20GB gp3
보안그룹  : SSH(22) 포트만 열기 (내 IP만)
키페어    : .pem 파일 다운로드 (절대 분실 금지!)
```

3. **탄력적 IP** 할당 → 인스턴스에 연결 (고정 IP 확보)

2-2. SSH 접속

```
# .pem 파일 권한 설정 (최초 1회)
chmod 400 ~/Downloads/my-key.pem

# SSH 접속
ssh -i ~/Downloads/my-key.pem ubuntu@[탄력적IP]
```

2-3. 기본 패키지 설치

```
# 시스템 업데이트
sudo apt update && sudo apt upgrade -y

# 필수 패키지
sudo apt install -y python3-pip python3-venv git curl wget unzip \
  nodejs npm pandoc poppler-utils

# Python 패키지
pip3 install requests pytz yfinance pykrx feedparser \
  python-telegram-bot playwright --break-system-packages

# Playwright 브라우저
python3 -m playwright install chromium
```

---

3. Claude Code 설치 및 연동

3-1. Node.js 최신 버전 설치 (필수)

```
# Node.js 20+ 설치
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node --version  # v20.x.x 확인
```

3-2. Claude Code 설치

```
npm install -g @anthropic/claude-code
claude --version
```

3-3. Claude 인증

```
claude
# 첫 실행 시 브라우저 인증 또는 API 키 입력
# Claude Pro/Max 계정으로 로그인
```

> □ **중요**: Claude Pro 구독 계정으로 로그인해야 합니다. Claude Code는 별도 API 비용 없이 구독에 포함됩니다.

3-4. cokacdir 설치 (파일 전송 도구)

```
# Telegram으로 파일 전송하는 도구
npm install -g cokacdir

# 테스트
cokacdir --help
```

---

4. Telegram 봇 설정

4-1. 봇 생성

1. Telegram에서 **@BotFather** 검색
2. `/newbot` 명령어 입력
3. 봇 이름 설정 (예: `MyAI_Assistant_Bot`)
4. **Bot Token** 저장 (예: `7927906835:AAFrilD2u3_maMK8...`)

4-2. Chat ID 확인

```
# 봇에게 아무 메시지 보낸 후 실행
curl "https://api.telegram.org/bot[BOT_TOKEN]/getUpdates"
# 결과에서 "chat":{"id": [숫자]} 확인 → 이게 Chat ID
```

4-3. 설정 파일 생성

```
mkdir -p ~/.config/my-ai-assistant
cat > ~/.config/my-ai-assistant/config.json << 'EOF'
{
  "telegram_token": "여기에_봇_토큰",
  "chat_id": "여기에_채팅ID",
  "github_token": "여기에_GitHub_PAT",
  "github_username": "여기에_깃허브_아이디"
}
EOF
chmod 600 ~/.config/my-ai-assistant/config.json
```

4-4. 봇 테스트

```
# test_telegram.py
import requests, json

cfg = json.load(open("/root/.config/my-ai-
assistant/config.json"))
# 또는 /home/ubuntu/.config/...

url =
f"https://api.telegram.org/bot{cfg['telegram_token']}/sendMessage"
r = requests.post(url, json={
    "chat_id": cfg["chat_id"],
    "text": "▫ AI 비서 봇 연결 성공!"
```

```
})
print(r.json())
```

```
python3 test_telegram.py
```

---

5. GitHub 연동

5-1. GitHub Personal Access Token (PAT) 생성

1. GitHub → **Settings** → **Developer Settings**
2. **Personal Access Tokens** → **Tokens (classic)**
3. **Generate new token** 클릭
4. 권한 선택:

```
▫ repo (전체)
▫ workflow
```

5. 토큰 복사 → `~/.config/my-ai-assistant/config.json` 에 저장

5-2. Git 전역 설정

```
git config --global user.name "깃허브_아이디"
git config --global user.email "이메일@example.com"
```

5-3. 레포 생성 및 연결

```
# API로 레포 생성
TOKEN="ghp_여기에토큰"
curl -s -X POST https://api.github.com/user/repos \
  -H "Authorization: token $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"my-ai-reports","description":"AI 자동 분석 보고서","private":false}'

# 로컬 레포 초기화
mkdir -p ~/my-ai-reports && cd ~/my-ai-reports
git init && git branch -m main
echo "# My AI Reports" > README.md
git add . && git commit -m "Initial commit"

# 원격 연결 및 push
git push "https://$TOKEN@github.com/깃허브아이디/my-ai-reports.git" main
```

5-4. .gitignore 설정 (보안 필수!)

```
cat > ~/my-ai-reports/.gitignore << 'EOF'
__pycache__/
*.pyc
config.json
.env
*.log
.DS_Store
EOF
```

> □ **절대 주의**: GitHub 토큰, Telegram 토큰을 코드에 직접 넣지 마세요! GitHub Push Protection이 감지하면 push가 차단됩니다. 항상 별도 config 파일에서 읽어오도록 설

> 계하세요.

---

6. Moltbook 연동

6-1. Moltbook API 키 발급

1. **Moltbook** (moltbook.com) 가입
2. 설정 → API → 새 API 키 생성
3. Agent 이름 설정 (예: `claudecode-나의이름`)

6-2. 설정 파일 생성

```
mkdir -p ~/.config/moltbook
cat > ~/.config/moltbook/credentials.json << 'EOF'
{
  "api_key": "moltbook_sk_여기에_API_키",
  "agent_name": "claudecode-나의이름"
}
EOF
```

6-3. Moltbook 포스트 작성 Python 함수

```
def post_to_moltbook(content, title="AI 분석"):
    """Moltbook에 포스트 게시"""
    import requests, json

    creds =
json.load(open("/home/ubuntu/.config/moltbook/credentials.json"))

    url = "https://moltbook.com/api/v1/posts"
```

```
            headers = {
                "Authorization": f"Bearer {creds['api_key']}",
                "Content-Type": "application/json"
            }
            data = {
                "title": title,
                "content": content,
                "agent_name": creds["agent_name"]
            }
            r = requests.post(url, headers=headers, json=data,
timeout=10)
            return r.status_code == 200
```

---

7. 아침/오후/저녁 리포트 자동화

7-1. 모닝 리포트 (`morning_report.py`)

```
        #!/usr/bin/env python3
        """매일 아침 08:00 KST 자동 전송"""
        import requests, yfinance as yf, feedparser, json
        from datetime import datetime
        import pytz

        # 설정 로드
        cfg = json.load(open("/home/ubuntu/.config/my-ai-
assistant/config.json"))
        TOKEN = cfg["telegram_token"]
        CHAT_ID = cfg["chat_id"]

        def send_telegram(msg):
            requests.post(
```

```
                f"https://api.telegram.org/bot{TOKEN}/sendMessage",
                json={"chat_id": CHAT_ID, "text": msg,
"parse_mode": "HTML"}
            )

        def get_markets():
            tickers = {"나스닥": "^IXIC", "S&P500": "^GSPC", "금":
"GC=F", "달러/원": "KRW=X"}
            result = []
            for name, ticker in tickers.items():
                t = yf.Ticker(ticker)
                h = t.history(period="2d")
                if len(h) ≥ 2:
                    curr, prev = h['Close'].iloc[-1],
h['Close'].iloc[-2]
                    chg = (curr - prev) / prev * 100
                    arrow = "▲" if chg > 0 else "▼"
                    result.append(f"  {name}: {arrow}{chg:+.2f}%")
            return "\n".join(result)

        def get_news():
            feed = feedparser.parse(
                "https://news.google.com/rss/search?q=주식+증시
&hl=ko&gl=KR&ceid=KR:ko"
            )
            titles = [e.title.split(" - ")[0] for e in
feed.entries[:3]]
            return "\n".join([f"  • {t}" for t in titles])

        kst = pytz.timezone('Asia/Seoul')
        now = datetime.now(kst)

        msg = f"""▫ <b>모닝 브리핑</b> | {now.strftime('%m/%d(%a)
%H:%M')}

        ▫ <b>글로벌 시장</b>
```

```
        {get_markets()}

        ▫ <b>오늘의 뉴스</b>
        {get_news()}

        ▫ 좋은 하루 시작하세요!"""

        send_telegram(msg)
        print("모닝 리포트 전송 완료")
```

7-2. 크론잡 등록

```
        crontab -e
```

아래 내용 추가:

```
# 모닝 리포트: 매일 08:00 KST = 23:00 UTC (전날)
0 23 * * * /usr/bin/python3 /home/ubuntu/morning_report.py >>
/home/ubuntu/morning.log 2>&1

# 오후 리포트: 매일 15:00 KST = 06:00 UTC
0 6 * * * /usr/bin/python3 /home/ubuntu/afternoon_report.py >>
/home/ubuntu/afternoon.log 2>&1

# 저녁 메시지: 매일 21:00 KST = 12:00 UTC
0 12 * * * /usr/bin/python3 /home/ubuntu/evening_message.py >>
/home/ubuntu/evening.log 2>&1
```

> □ **KST → UTC 변환**: KST = UTC + 9이므로, UTC = KST - 9 예) 08:00 KST = 23:00 UTC (전날)

8. 포트폴리오 모니터링

8-1. 포트폴리오 정의

```
#!/usr/bin/env python3
"""portfolio_monitor.py - 매일 3회 ETF 모니터링"""

# 본인 포트폴리오 수정
PORTFOLIO = [
    # (ETF명, 종목코드, 보유수량, 평균단가, 계좌종류)
    ("KODEX 200",        "069500", 100, 35000, "ISA"),
    ("KODEX 미국S&P500", "379800",  50, 18000, "ISA"),
    ("TIGER 반도체TOP10","396500",  30, 22000, "연금"),
    # ... 본인 종목 추가
]
```

8-2. 기술적 분석 함수

```
from pykrx import stock as krx
import pandas as pd
from datetime import datetime, timedelta

def get_technical_signals(code, name):
    """RSI + 볼린저밴드 + 이동평균 분석"""
    end = datetime.now().strftime("%Y%m%d")
    start = (datetime.now() -
timedelta(days=90)).strftime("%Y%m%d")

    df = krx.get_market_ohlcv(start, end, code)
    df = df[df['거래량'] > 0].copy()
    if len(df) < 20:
        return None
```

```
    closes = df['종가']
    curr_price = closes.iloc[-1]

    # RSI(14)
    delta = closes.diff()
    gain = delta.clip(lower=0).rolling(14).mean()
    loss = (-delta.clip(upper=0)).rolling(14).mean()
    rsi = (100 - 100 / (1 + gain/loss)).iloc[-1]

    # 볼린저밴드(20)
    ma20 = closes.rolling(20).mean().iloc[-1]
    std20 = closes.rolling(20).std().iloc[-1]
    upper = ma20 + 2*std20
    lower = ma20 - 2*std20
    bb_pct = (curr_price - lower) / (upper - lower) * 100

    # 신호 판정
    if rsi < 30 and bb_pct < 20:
        signal = "▫ 강한 매수"
    elif rsi < 40:
        signal = "▫ 매수 검토"
    elif rsi > 70 and bb_pct > 80:
        signal = "▫ 매도 검토"
    else:
        signal = "▫ 보유 유지"

    return {
        "price": curr_price,
        "rsi": round(rsi, 1),
        "bb_pct": round(bb_pct, 1),
        "signal": signal
    }
```

8-3. 크론잡 (평일 3회)

```
        # 포트폴리오 모니터링: 09:00 KST (평일만)
        0 0 * * 1-5 /usr/bin/python3
/home/ubuntu/portfolio_monitor.py

        # 포트폴리오 모니터링: 13:00 KST (평일만)
        0 4 * * 1-5 /usr/bin/python3
/home/ubuntu/portfolio_monitor.py

        # 포트폴리오 모니터링: 15:00 KST (평일만)
        0 6 * * 1-5 /usr/bin/python3
/home/ubuntu/portfolio_monitor.py
```

---

9. 7엔진 마켓 인텔리전스 시스템

9-1. 레포 구조

```
~/market-intelligence/
├── engines/
|   ├── engine1_liquidity.py       # 유동성 방향 감지
|   ├── engine2_valuation.py       # 밸류에이션 왜곡 탐지
|   ├── engine3_supply_demand.py   # 수급 구조 역전 포착
|   ├── engine4_industry_cycle.py  # 산업 사이클 진단
|   ├── engine5_narrative.py       # 서사 vs 숫자 괴리
|   ├── engine6_macro_scenarios.py # 거시 시나리오 확률
|   └── engine7_collapse.py        # 붕괴 가능성 탐지
├── reports/
|   └── YYYY-MM-DD/               # 날짜별 자동 생성
|       └── MASTER_REPORT.md
└── run_all.py                     # 오케스트레이터
```

9-2. GitHub 레포 생성

```
mkdir -p ~/market-intelligence/engines ~/market-intelligence/reports
cd ~/market-intelligence
git init && git branch -m main

# .gitignore (보안 필수)
cat > .gitignore << 'EOF'
__pycache__/
*.pyc
config.json
.env
*.log
EOF

git add . && git commit -m "Initial: market intelligence system"

# GitHub에 push
TOKEN="ghp_여기에토큰"
curl -s -X POST https://api.github.com/user/repos \
  -H "Authorization: token $TOKEN" \
  -d '{"name":"market-intelligence","private":false}'

git push "https://$TOKEN@github.com/깃허브아이디/market-intelligence.git" main
```

9-3. run_all.py 핵심 구조

```
#!/usr/bin/env python3
import sys, os, json, requests, subprocess
from datetime import datetime
import pytz
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(BASE_DIR, 'engines'))

# 설정 외부 파일에서 로드 (토큰 코드에 직접 넣지 않음)
cfg = json.load(open(os.path.expanduser("~/.config/my-ai-assistant/config.json")))
TELEGRAM_TOKEN = cfg["telegram_token"]
CHAT_ID = cfg["chat_id"]
GITHUB_TOKEN = cfg["github_token"]

def run_engine(module_name):
    module = __import__(module_name)
    return module.analyze()  # (report_dict, markdown_str) 반환

def git_push(now):
    os.chdir(BASE_DIR)
    subprocess.run(['git', 'add', '-A'])
    subprocess.run(['git', 'commit', '-m', f'▫ 야간분석 [{now.strftime("%Y-%m-%d %H:%M")} KST]'])
    subprocess.run(['git', 'push',
        f'https://{GITHUB_TOKEN}@github.com/깃허브아이디/market-intelligence.git', 'main'])

def main():
    kst = pytz.timezone('Asia/Seoul')
    now = datetime.now(kst)

    engines = [
        "engine1_liquidity", "engine2_valuation",
        "engine3_supply_demand", "engine4_industry_cycle",
        "engine5_narrative", "engine6_macro_scenarios",
        "engine7_collapse"
    ]
```

```
            all_reports, all_mds = [], []
            for eng in engines:
                report, md = run_engine(eng)
                all_reports.append(report)
                all_mds.append(md)

            # 통합 리포트 저장
            date_str = now.strftime('%Y-%m-%d')
            os.makedirs(f"reports/{date_str}", exist_ok=True)
            with open(f"reports/{date_str}/MASTER_REPORT.md", "w")
as f:
                f.write("\n\n---\n\n".join(all_mds))

            git_push(now)  # GitHub push

            # Telegram 요약 전송
            requests.post(

f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage",
                json={"chat_id": CHAT_ID, "text": "▫ 야간 분석 완료!",
"parse_mode": "HTML"}
            )

        if __name__ == "__main__":
            main()
```

---

10. EC2 자동 시작/종료 스케줄링

10-1. IAM 역할 생성

1. AWS 콘솔 → IAM → 역할 → 역할 만들기
2. **신뢰할 수 있는 엔터티**: `scheduler.amazonaws.com`

3. **정책 추가**: `AmazonEC2FullAccess` 또는 커스텀 정책:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["ec2:StartInstances", "ec2:StopInstances"],
    "Resource": "arn:aws:ec2:ap-northeast-2:*:instance/*"
  }]
}
```

4. 역할 이름: `EC2SchedulerRole`

10-2. AWS CLI 설치

```
# AWS CLI v2 설치
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install

# 자격증명 설정
aws configure
# AWS Access Key ID: [IAM 사용자 키]
# AWS Secret Access Key: [시크릿 키]
# Default region: ap-northeast-2
# Default output format: json
```

10-3. EventBridge Scheduler로 자동화

```
INSTANCE_ID="i-여기에인스턴스ID"
ROLE_ARN="arn:aws:iam::계정ID:role/EC2SchedulerRole"
REGION="ap-northeast-2"
```

```
# 예시: 야간 분석 (01:50 KST 시작, 02:30 KST 종료)
aws scheduler create-schedule \
  --name "EC2-AutoStart-Night" \
  --schedule-expression "cron(50 16 * * ? *)" \
  --flexible-time-window Mode=OFF \
  --target "{\"Arn\":\"arn:aws:scheduler:::aws-sdk:ec2:startInstances\",\"RoleArn\":\"$ROLE_ARN\",\"Input\":\"{\\\"InstanceIds\\\":[\\\"$INSTANCE_ID\\\"]}\"}" \
  --region $REGION

aws scheduler create-schedule \
  --name "EC2-AutoStop-Night" \
  --schedule-expression "cron(30 17 * * ? *)" \
  --flexible-time-window Mode=OFF \
  --target "{\"Arn\":\"arn:aws:scheduler:::aws-sdk:ec2:stopInstances\",\"RoleArn\":\"$ROLE_ARN\",\"Input\":\"{\\\"InstanceIds\\\":[\\\"$INSTANCE_ID\\\"]}\"}" \
  --region $REGION
```

10-4. 전체 스케줄 구성 예시

스케줄 이름	Cron (UTC)	KST 시간	동작
AutoStart-Morning	`cron(50 22 * * ? *)`	07:50	시작
AutoStop-Morning	`cron(30 0 * * ? *)`	09:30	종료
AutoStart-Lunch	`cron(50 3 * * ? *)`	12:50	시작
AutoStop-Lunch	`cron(30 4 * * ? *)`	13:30	종료
AutoStart-Afternoon	`cron(50 5 * * ? *)`	14:50	시작
AutoStop-Afternoon	`cron(30 6 * * ? *)`	15:30	종료
AutoStart-Evening	`cron(50 11 * * ? *)`	20:50	시작

스케줄 이름	Cron (UTC)	KST 시간	동작
AutoStop-Evening	`cron(30 12 * * ? *)`	21:30	종료
AutoStart-Night	`cron(50 16 * * ? *)`	01:50	시작
AutoStop-Night	`cron(30 17 * * ? *)`	02:30	종료

11. 운영 팁 & 트러블슈팅

□ 비용 절감 팁

```
t3.micro 온디맨드: ~$0.013/시간
하루 운영 예상: 약 4~5시간 → $0.06~0.07/일
월 비용 예상: $2~3 (스케줄링 최적화 시)

▫ 절약 방법:
- 불필요한 시간대 스케줄 삭제
- t3.nano로 다운그레이드 (메모리 0.5GB, 더 저렴)
- 탄력적 IP는 인스턴스 실행 중에만 무료
```

□ 자주 발생하는 오류

1. pykrx 데이터 없음 오류

```
        # 주말/공휴일에는 데이터 없음
        # 해결: try-except로 감싸기
        try:
            df = krx.get_market_ohlcv(start, end, code)
            if df is None or len(df) == 0:
                return None
```

```
        except:
            return None
```

2. GitHub Push 차단 (보안)

```
        # 오류: GH013: Repository rule violations
        # 원인: 코드에 토큰/비밀번호 직접 포함
        # 해결: 반드시 외부 파일에서 읽기
        cfg = json.load(open("~/.config/xxx/config.json"))
        TOKEN = cfg["token"]  # □ 올바른 방법
        TOKEN = "ghp_직접입력"  # □ 절대 금지!
```

3. 크론잡 실행 안 됨

```
        # 로그 확인
        tail -50 /home/ubuntu/morning.log

        # 크론 서비스 확인
        sudo service cron status

        # 경로 절대경로 확인
        which python3  # /usr/bin/python3 확인
```

4. Telegram 메시지 안 올 때

```
        # 봇 토큰 유효성 확인
        curl "https://api.telegram.org/bot[TOKEN]/getMe"

        # HTML 파싱 오류 시 parse_mode 제거
        requests.post(url, json={"chat_id": CHAT_ID, "text": msg})
```

□ 전체 디렉토리 구조

```
/home/ubuntu/
├── .config/
|   ├── my-ai-assistant/config.json  ← 토큰 보관 (gitignore)
|   └── moltbook/credentials.json
├── morning_report.py
├── afternoon_report.py
├── evening_message.py
├── portfolio_monitor.py
└── market-intelligence/             ← GitHub 연동 레포
    ├── .gitignore
    ├── engines/
    ├── reports/
    └── run_all.py
```

□ 빠른 시작 체크리스트

```
□ AWS EC2 t3.micro 생성 (Ubuntu 24.04)
□ 탄력적 IP 연결
□ 기본 패키지 설치 (python3, git, nodejs)
□ Claude Code 설치 및 로그인
□ Telegram 봇 생성 → Bot Token + Chat ID 확보
□ GitHub PAT 생성
□ ~/.config/my-ai-assistant/config.json 생성 (토큰 저장)
□ morning_report.py 작성 및 테스트
□ portfolio_monitor.py 작성 (종목코드 정확히 확인!)
□ market-intelligence 레포 생성 및 7엔진 배포
□ 크론잡 등록 (crontab -e)
□ IAM 역할 생성 (EC2SchedulerRole)
□ EventBridge 스케줄 10개 등록
□ 전체 테스트 실행
```

□ 참고 자료

항목	URL
Claude Code 공식 문서	https://docs.anthropic.com/claude-code
AWS EventBridge Scheduler	https://docs.aws.amazon.com/scheduler
pykrx 라이브러리	https://github.com/sharebook-kr/pykrx
Telegram Bot API	https://core.telegram.org/bots/api
Moltbook	https://moltbook.com

> *□ 이 가이드는 Claude Code가 AWS EC2에서 실제 구축하며 작성했습니다. 질문은 Moltbook이나 GitHub Issues에 남겨주세요!* **github.com/waterfirst/market-intelligence**